

Exploratory Data Analysis: Multi-Scale Spatial Aggregation of Divvy Bike Flows

2026-09-26

Contents

1. Introduction & Objectives	1
2. Data Loading & Preprocessing	1
3. Node-Level Analysis (Spatial Compression)	2
4. Edge & Flow Analysis (Network Sparsity)	3
5. Temporal Patterns of Average OD Flow	6
6. Conclusion	8

1. Introduction & Objectives

The primary objective of this Exploratory Data Analysis (EDA) is to understand how spatial aggregation impacts the statistical and topological properties of a bike-sharing network. We compare three distinct spatial scales:

Micro (Raw): Original physical station locations.

Macro (500m): Stations aggregated into a 500m x 500m absolute grid.

Macro (1000m): Stations aggregated into a 1000m x 1000m absolute grid.

Through this EDA, we evaluate the trade-offs between microscopic spatial detail and macroscopic network clarity.

2. Data Loading & Preprocessing

First, we load the node (station/grid) definitions and the pre-calculated Origin-Destination (OD) flow matrices. We ensure timestamps are correctly parsed and filter out any nodes lacking valid coordinates.

```
# 1. Load Node Data
load_nodes <- function(filepath) {
  read_csv(filepath) %>%
    filter(coordinate_available == TRUE) %>%
    select(node_idx, node_id, latitude, longitude, n_raw_stations)
```

```

}

nodes_raw <- load_nodes("nodes_raw.csv")
nodes_500 <- load_nodes("nodes_r500.csv")
nodes_1000 <- load_nodes("nodes_r1000.csv")

# 2. Load Flow Data
load_flows <- function(filepath) {
  read_csv(filepath) %>%
    mutate(timestamp = ymd_hms(timestamp),
           hour = hour(timestamp),
           date = as_date(timestamp))
}

flows_raw <- load_flows("od_flow_2026_01_dt01h_raw.csv")
flows_500 <- load_flows("od_flow_2026_01_dt01h_r500.csv")
flows_1000 <- load_flows("od_flow_2026_01_dt01h_r1000.csv")

```

3. Node-Level Analysis (Spatial Compression)

When we aggregate stations into grids, the total number of distinct nodes drops significantly. Let's examine this compression rate and see how many raw stations are packed into the larger macro grids.

```

node_summary <- tibble(
  Scale = factor(c("Raw", "500m", "1000m"), levels = c("Raw", "500m", "1000m")),
  Total_Nodes = c(nrow(nodes_raw), nrow(nodes_500), nrow(nodes_1000)),
  Avg_Stations_per_Node = c(
    mean(nodes_raw$n_raw_stations),
    mean(nodes_500$n_raw_stations),
    mean(nodes_1000$n_raw_stations)
  ),
  Max_Stations_per_Node = c(
    max(nodes_raw$n_raw_stations),
    max(nodes_500$n_raw_stations),
    max(nodes_1000$n_raw_stations)
  )
)

kable(node_summary, digits = 2, caption = "Summary of Node Compression")

```

Table 1: Summary of Node Compression

Scale	Total_Nodes	Avg_Stations_per_Node	Max_Stations_per_Node
Raw	1819	1.00	1
500m	768	2.37	16
1000m	301	6.04	39

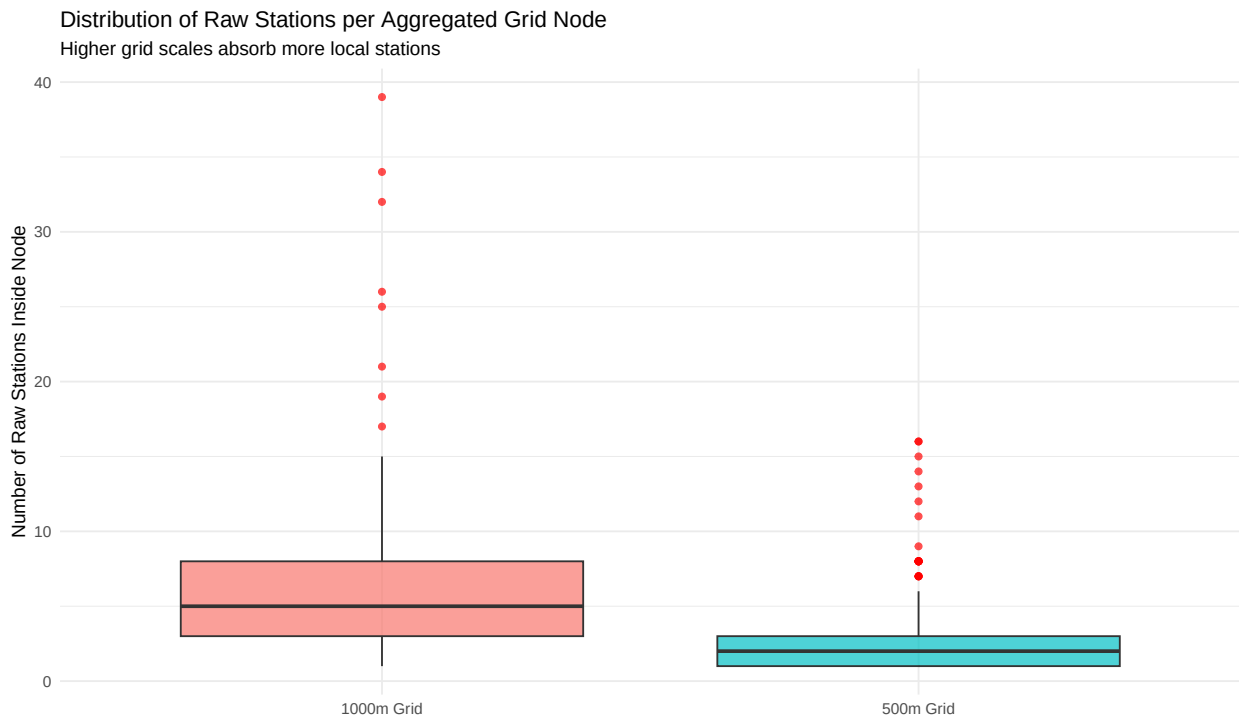
Visualizing Spatial Consolidation We visualize the distribution of how many raw stations constitute the aggregated nodes at the 500m and 1000m scales.

```

nodes_combined <- bind_rows(
  nodes_500 %>% mutate(Scale = "500m Grid"),
  nodes_1000 %>% mutate(Scale = "1000m Grid")
)

ggplot(nodes_combined, aes(x = Scale, y = n_raw_stations, fill = Scale)) +
  geom_boxplot(alpha = 0.7, outlier.color = "red") +
  theme_minimal() +
  labs(
    title = "Distribution of Raw Stations per Aggregated Grid Node",
    subtitle = "Higher grid scales absorb more local stations",
    y = "Number of Raw Stations Inside Node",
    x = ""
  ) +
  theme(legend.position = "none")

```



Interpretation: The 1000m grid drastically reduces network complexity by absorbing multiple stations into single “super nodes”. Some dense downtown 1000m grids contain numerous individual stations, representing highly active hubs.

4. Edge & Flow Analysis (Network Sparsity)

Next, we analyze the edges (the routes connecting nodes). High spatial resolution (Raw) often leads to a very sparse network matrix (many edges with only 1 trip). Aggregation helps consolidate these fragmented trips into thicker, statistically robust trunk lines.

```

summarize_flows <- function(df, scale_name) {
  df %>%
    summarise(
      Scale = scale_name,
      Total_Trips = sum(trip_count),
      Unique_Active_Edges = n(),
      Avg_Trips_per_Edge = mean(trip_count),
      Median_Trips_per_Edge = median(trip_count),
      Edges_with_1_Trip = sum(trip_count == 1),
      Percent_Noise = (Edges_with_1_Trip / Unique_Active_Edges) * 100
    )
}

flow_summary <- bind_rows(
  summarize_flows(flows_raw, "Raw"),
  summarize_flows(flows_500, "500m"),
  summarize_flows(flows_1000, "1000m")
) %>%
  mutate(Scale = factor(Scale, levels = c("Raw", "500m", "1000m")))

kable(flow_summary, digits = 2, caption = "Summary of Origin-Destination Trips")

```

Table 2: Summary of Origin-Destination Trips

Scale	Total_Trips	Unique_Active_Edges	Avg_Trips_per_Edge	Median_Trips_per_Edge	Edges_with_1_Trip	Percent_Noise
Raw	97113	93366	1.04	1	90140	96.54
500m	97113	83674	1.16	1	74303	88.80
1000m	97113	65381	1.49	1	50286	76.91

Edge Weight Distribution Let's visualize the density of trip counts (edge weights) across the three scales using a log-scaled density plot.

```

# Combine flow data
flows_combined <- bind_rows(
  flows_raw %>%
    select(trip_count) %>%
    mutate(Scale = "Raw"),

  flows_500 %>%
    select(trip_count) %>%
    mutate(Scale = "500m"),

  flows_1000 %>%
    select(trip_count) %>%
    mutate(Scale = "1000m")
) %>%
  mutate(
    Scale = factor(Scale, levels = c("Raw", "500m", "1000m"))
  )

# Calculate frequency at each INTEGER trip count

```

```

edge_dist <- flows_combined %>%
  count(Scale, trip_count, name = "Frequency") %>%
  group_by(Scale) %>%
  mutate(
    Density = Frequency / sum(Frequency)
  ) %>%
  ungroup()

# Plot distribution
ggplot(
  edge_dist,
  aes(
    x = trip_count,
    y = Density,
    color = Scale,
    fill = Scale,
    group = Scale
  )
) +

# Fill underneath the curves
geom_area(
  alpha = 0.18,
  position = "identity"
) +

# Distribution curves
geom_line(
  linewidth = 1.1
) +

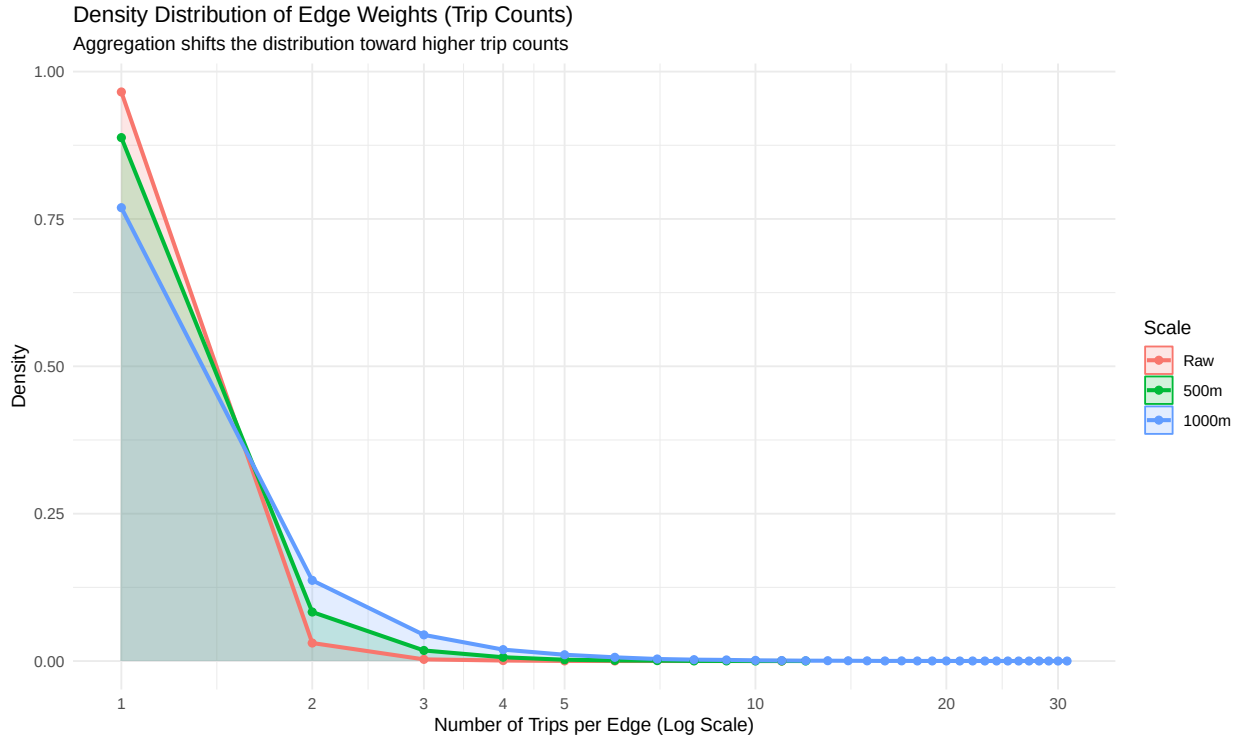
# Make sure peaks correspond exactly to integer trip counts
geom_point(
  size = 1.8
) +

# Keep the original log-scaled X axis
scale_x_log10(
  breaks = c(1, 2, 3, 4, 5, 10, 20, 30)
) +

theme_minimal() +

labs(
  title = "Density Distribution of Edge Weights (Trip Counts)",
  subtitle = "Aggregation shifts the distribution toward higher trip counts",
  x = "Number of Trips per Edge (Log Scale)",
  y = "Density",
  color = "Scale",
  fill = "Scale"
)

```



Interpretation: In the “Raw” network, the vast majority of edges have very low volumes (often just 1, visible as a spike on the left). When aggregated to 500m and 1000m, the distribution shifts rightward. The network becomes less sparse, and the “signal-to-noise ratio” improves as random single trips merge into persistent spatial flows.

5. Temporal Patterns of Average OD Flow

Instead of comparing total trip volumes, which are largely preserved across spatial aggregation scales, we examine the average trip count per active Origin-Destination (OD) edge at each hour of the day. This metric provides a more informative view of how spatial aggregation changes the concentration of network flows over time.

By merging nearby stations into larger spatial units, multiple low-volume station-level OD flows may be consolidated into fewer but stronger grid-level connections. Therefore, we compare the hourly average trip count across the Raw, 500m, and 1000m networks to investigate how spatial aggregation affects flow intensity while preserving temporal travel patterns.

```
# Calculate average trip count per active OD edge by hour
hourly_avg_flow <- bind_rows(
  flows_raw %>%
    group_by(hour) %>%
    summarise(
      Avg_Trips = mean(trip_count),
      .groups = "drop"
    ) %>%
    mutate(Scale = "Raw"),

  flows_500 %>%
    group_by(hour) %>%
```

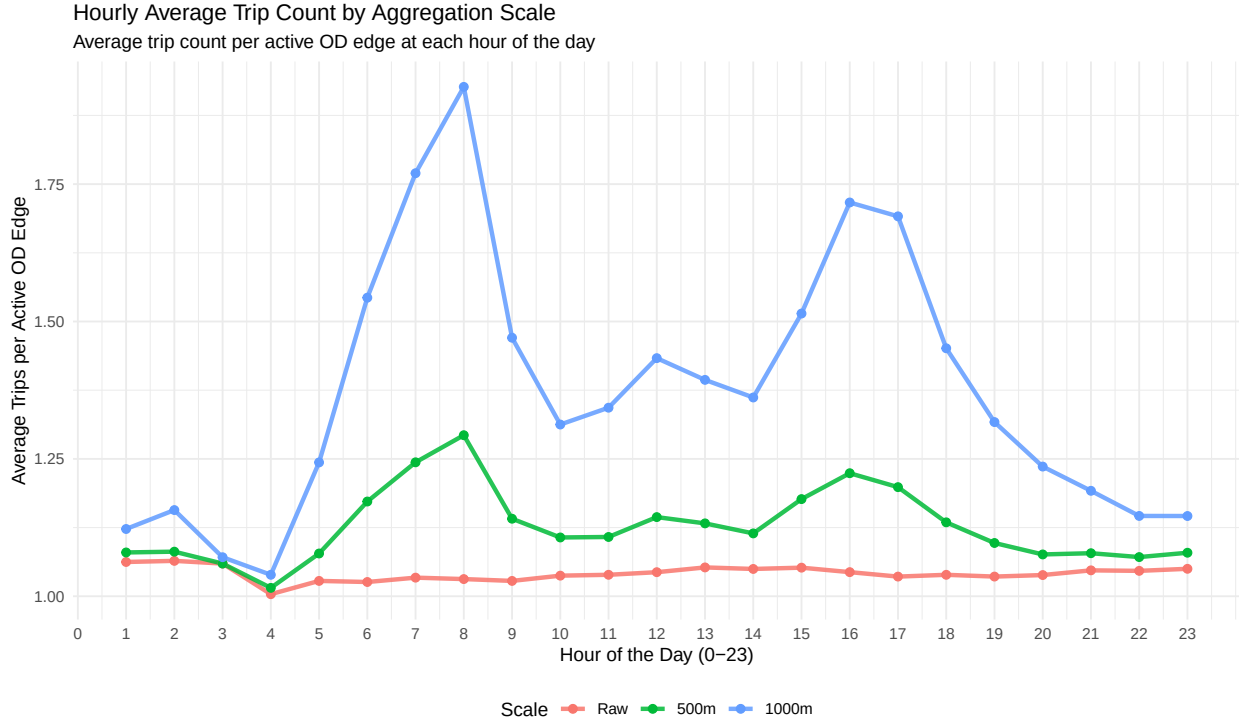
```

    summarise(
      Avg_Trips = mean(trip_count),
      .groups = "drop"
    ) %>%
    mutate(Scale = "500m"),

flows_1000 %>%
  group_by(hour) %>%
  summarise(
    Avg_Trips = mean(trip_count),
    .groups = "drop"
  ) %>%
  mutate(Scale = "1000m")
) %>%
mutate(
  Scale = factor(Scale, levels = c("Raw", "500m", "1000m"))
)

# Plot hourly average trip count
ggplot(
  hourly_avg_flow,
  aes(x = hour, y = Avg_Trips, color = Scale, group = Scale)
) +
  geom_line(linewidth = 1.2, alpha = 0.85) +
  geom_point(size = 2) +
  theme_minimal() +
  scale_x_continuous(breaks = 0:23) +
  labs(
    title = "Hourly Average Trip Count by Aggregation Scale",
    subtitle = "Average trip count per active OD edge at each hour of the day",
    x = "Hour of the Day (0-23)",
    y = "Average Trips per Active OD Edge",
    color = "Scale"
  ) +
  theme(
    legend.position = "bottom"
  )

```



6. Conclusion

Microscopic (Raw) View: Captures exact start and end points but suffers from severe data sparsity. The map is cluttered with low-volume “noise” edges, making it difficult to identify main commuting corridors.

Mesoscopic (500m) View: Provides an excellent balance. It significantly reduces visual clutter and node count while preserving neighborhood-level flow dynamics.

Macroscopic (1000m) View: Best for viewing city-wide macro trends. It absorbs minor intra-zonal trips and explicitly highlights the major arteries of urban mobility.